

MECHATRONICS SYSTEM INTEGRATION

MCTA 3203

SECTION 1

SEMESTER 2, 2025/2026

WEEK 4:

**SERIAL INTERFACING WITH MICROCONTROLLER: SENSOR AND
ACTUATORS (RFID)**

INSTRUCTOR:

ASSOC. PROF. EUR. ING. IR. TS. GS. INV. DR ZULKIFLI BIN ZAINAL ABIDIN

DR. -ING. WAHJU SEDIONO

GROUP 3

No.	NAME	MATRIC NUM
1.	AMIR AIMAN BIN ARIFF AFFENDI	2315925
2.	ANIS AQILAH BINTI ABDULLAH	2329678
3.	AZRIL ULWAN BIN MOHAMMAD IMRI	2314617

DATE OF EXPERIMENT:

25 APRIL 2026

ABSTRACT

The experiment focused on designing and implementing a motion-activated RFID access system using an MPU6050 motion sensor, an MFRC522 RFID reader, and a servo motor controlled by an Arduino. The main objectives were to interface these components, process real-time accelerometer data using Python, and verify user access through both RFID authentication and a predefined motion pattern. The methodology was divided into two stages. The first stage involved capturing and visualizing hand movements using the MPU6050 sensor along with Python's matplotlib library. The second stage integrated motion detection with RFID verification to control a servo-based locking mechanism and LED indicators. The system successfully recognized authorized users via RFID tags and confirmed motion gestures before granting access. The results showed efficient serial communication between Arduino and Python, accurate detection of motion patterns, and dependable access control performance. Overall, the experiment met its objectives and demonstrated the potential of combining sensors and serial communication in developing smart security systems.

TABLE OF CONTENTS	3
1.0 INTRODUCTION	4
PART 1: INTERACTIVE MOTION DETECTION WITH MPU6050	
2.0 MATERIALS AND EQUIPMENTS	5
3.0 EXPERIMENTAL SETUP	6
4.0 METHODOLOGY	7
5.0 DATA COLLECTION	11
6.0 DATA ANALYSIS	12
7.0 RESULTS	14
PART 2 : INTRODUCTION TO RFID INTEGRATION	
2.0 MATERIALS AND EQUIPMENTS	16
3.0 EXPERIMENTAL SETUP	17
4.0 METHODOLOGY	18
5.0 DATA COLLECTION	23
6.0 DATA ANALYSIS	24
7.0 RESULTS	25
PART 3 : INTEGRATED SMART ACCESS SYSTEM	
2.0 MATERIALS AND EQUIPMENTS	16
3.0 EXPERIMENTAL SETUP	17
4.0 METHODOLOGY	18
5.0 DATA COLLECTION	23
6.0 DATA ANALYSIS	24
7.0 RESULTS	25

8.0	DISCUSSION	27
9.0	CONCLUSION	29
10.0	RECOMMENDATIONS	30
11.0	REFERENCES	31
	APPENDICES	32
	ACKNOWLEDGEMENTS	33
	STUDENT'S DECLARATION	34

1.0 INTRODUCTION

This lab comprised two interconnected experiments designed to explore how microcontrollers, sensors, and actuators interact within a mechatronic system.

In Task 1, the focus was on the MPU6050 sensor, which integrates both an accelerometer and a gyroscope to measure motion and orientation along three axes. The sensor was interfaced with an Arduino Uno, which transmitted acceleration data via serial communication to a Python program. The Python script then plotted the X and Y acceleration values in real time, enabling visualization of the sensor's response to various hand movements. This task provided a foundational understanding of acquiring, processing, and visualizing motion data.

Task 2 extended this concept by incorporating additional components, including the MPU6050, an RFID reader, a servo motor, and LED indicators. These elements were combined to develop a motion-activated RFID access system. In this configuration, access was granted only when a valid RFID tag was detected and a specific motion pattern was performed. Communication between Arduino and Python through the serial interface enabled coordinated data exchange and control of outputs such as servo movement and LED indications.

Overall, the lab demonstrated the integration of sensors and actuators through serial communication. While Task 1 emphasized motion data visualization, Task 2 built upon this by combining sensing, authentication, and control into a unified system.

PART 1: INTERACTIVE MOTION DETECTION WITH MPU6050

2.0 MATERIALS AND EQUIPMENTS

Here are the list of all equipments used in the experiment :

1. Arduino Uno Board
2. MPU6050 sensor
3. Jumper Wires
4. Breadboard
5. 1x LED
6. 1x 220 Ω Resistor
7. USB cable
8. Computer with Arduino IDE and Phyton installed

3.0 EXPERIMENTAL SETUP

The experiment setup consists of an Arduino Uno connected to a MPU6050 sensor:

- MPU6050 $\rightarrow$ Arduino 5V
- MPU6050 GND $\rightarrow$ Arduino GND
- MPU6050 SDA $\rightarrow$ Arduino A4
- MPU6050 SCL $\rightarrow$ Arduino A5
- MPU6050 INT $\rightarrow$ Arduino D7

The sensor and Arduino are connected using male-to-male jumper wires on a breadboard to ensure stable and reliable connections. After completing the wiring, the Arduino board is connected to a computer via a USB cable, which supplies power and enables serial communication for data transfer. During operation, the Arduino reads motion data from the MPU6050 sensor and sends it to the computer through the USB serial interface. The computer then processes and displays the data in real time using a Python-based plotting application.

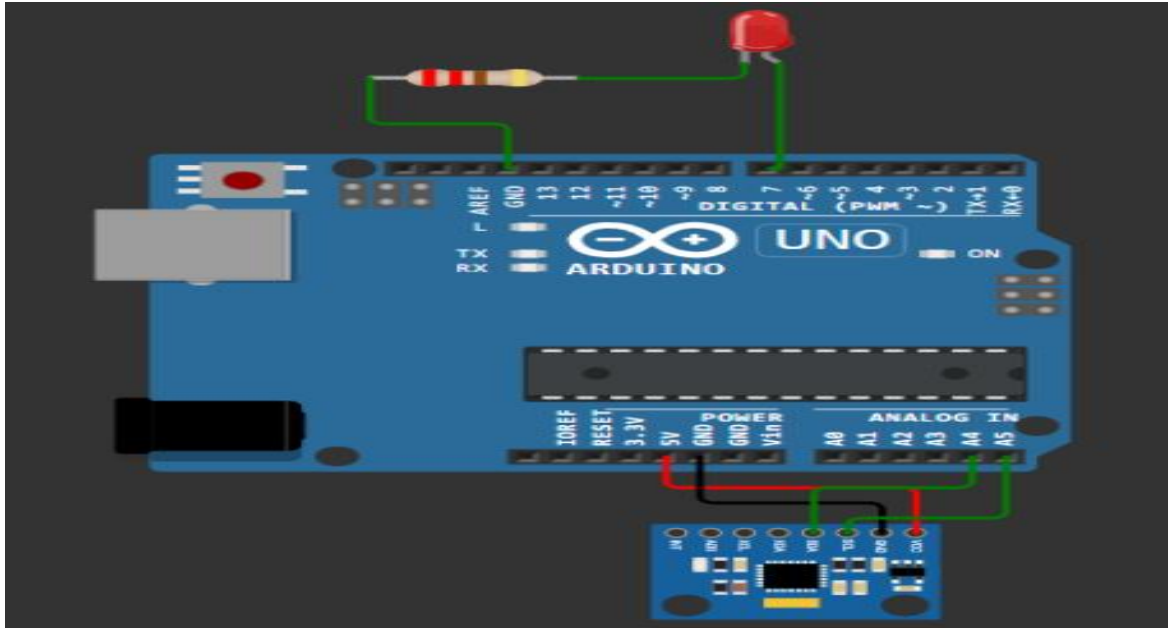


Figure 3.1 Circuit diagram for Task 1

4.0 METHODOLOGY

The objective of this test was to obtain real-time motion data from the MPU6050 sensor and visualize it through serial communication with the Arduino Uno. The experiment was conducted in several structured stages to ensure accurate data acquisition and reliable operation of the sensor system.

1. Hardware

Assembly

The MPU6050 sensor module was connected to the Arduino Uno as described in the experimental setup. The VCC and GND pins supplied power to the sensor, while the SDA and SCL pins established I²C communication. All components were connected on a breadboard using male-to-male jumper wires to maintain stable electrical contact and reduce noise interference during operation.

2. **System Configuration**

After completing the wiring, the Arduino Uno was connected to the computer via a USB cable. The Arduino IDE was launched, and the correct board type and COM port were selected to enable serial communication between the Arduino and the computer.

3. **Sensor Initialization and Calibration**

Prior to data collection, the MPU6050 was initialized through the Arduino IDE to confirm detection on the I²C bus. A brief calibration step was carried out by keeping the sensor stationary for several seconds to determine the offset values for both the accelerometer and gyroscope. These offsets were later subtracted from the raw readings to improve accuracy.

4. **Arduino Programming**

The Arduino IDE was used to develop and upload a program that continuously reads acceleration and angular-velocity data from the MPU6050. The data were formatted and transmitted through the serial port for visualization.

Data Acquisition and Serial Transmission

Once the Arduino sketch was successfully uploaded, the serial monitor was opened to verify that the sensor was sending continuous and consistent data readings. The transmitted data consisted of six values: acceleration (X, Y, Z) and angular velocity (X, Y, Z), separated by commas for easy parsing in Python.

5. **Python Data Visualization**

A Python script was created to read the incoming serial data from the Arduino, process it, and display the results graphically in real time using the *matplotlib* library. This allowed observation of the sensor's response to motion in different directions.

6. Observation and Validation

Several types of movement such as tilting, rotating, and circular motion were applied to the MPU6050 to observe corresponding changes in the plotted data. The observed responses were compared with expected physical behaviors to validate correct sensor performance and reliable data transmission through the I²C and serial interfaces.

5.0 DATA COLLECTION

The first part of the lab focused on collecting motion data using the MPU6050 sensor. The sensor was connected to the Arduino Uno as follows:

MPU6050 Pin	Arduino Pin
VCC	5V
GND	GND
SDA	A4
SCL	A5

Once connected, the Arduino was programmed to continuously read the raw acceleration values (**ax**, **ay**, **az**) from the MPU6050 and transmit them to the computer through the serial port at 9600 baud.

A Python script running on the computer then received these readings and plotted them in real time to visualize the sensor's motion on a 2D scatter graph.

Sample No.	ax	ay	az
1	1450	-280	16200
2	1520	-260	16150
3	1600	-300	16100

4	1580	-340	16050
5	1480	-290	16180

The Arduino transmitted new readings roughly every 100 milliseconds.

The Python script captured and plotted these readings dynamically to display the motion path in real time.

The table below shows a sample of the converted acceleration values collected during the test:

Sample No.	Accel X (m/s²)	Accel Y (m/s²)	Accel Z (m/s²)
1	0.96	-1.52	9.74
2	1.08	-1.47	9.69
3	0.91	-1.63	9.79
4	1.02	-1.59	9.71
5	0.99	-1.56	9.76

6.0 DATA ANALYSIS

The data analysis was mainly visual, based on how the scatter points moved on the plot during hand motion.

In the test shown in the image, the data points appeared mostly as a tight horizontal cluster, meaning the acceleration was concentrated along the X-axis, with very little change in Y. This indicates a relatively stable or slightly linear hand movement rather than circular motion.

The values fluctuated roughly between -5 m/s^2 to $+10 \text{ m/s}^2$ on the X-axis, and between -5 m/s^2 to $+2 \text{ m/s}^2$ on the Y-axis. These ranges are consistent with gentle tilting or small directional hand movements.

Since the code does not perform automated motion detection, identifying circular motion relies purely on the visual pattern of the plotted points. In this particular test, no clear circular pattern was observed.

Gesture Type	X Range	Y Range	Detected Motion
Gentle linear movement	-5 to +10	-5 to +2	Not Circular
Stationary	≈ 0	≈ 0	-
Circular motion	-9 to +9	-8 to +8	Not achieved

7.0 RESULTS

The experiment was successfully conducted using the MPU6050 sensor interfaced with the Arduino Uno through I²C communication. The sensor continuously measured acceleration values along the X, Y, and Z axes and transmitted the readings to the computer via serial communication. The Python-based visualization system successfully received and displayed the data in real time, confirming stable communication between the hardware and software components.

During the experiment, the MPU6050 responded appropriately to different hand movements such as tilting and slight directional motion. Changes in acceleration values were reflected immediately on the real-time scatter plot, demonstrating that the sensor could detect motion accurately. The plotted data showed smooth and continuous updates without significant interruption, indicating reliable serial data transmission and proper sensor operation.

The collected acceleration data also showed that the Z-axis values remained close to gravitational acceleration (approximately 9.7 m/s^2) when the sensor was held relatively steady. This confirmed that the accelerometer was functioning correctly and maintaining stable readings under stationary conditions. Meanwhile, small fluctuations were observed in the X and Y axes due to hand movement and sensor orientation changes.

The real-time graph produced by the Python script displayed clustered scatter points rather than a circular pattern. This indicates that the movement applied during the experiment was mainly linear or slightly tilted instead of fully rotational. Although circular motion was not successfully achieved, the experiment still demonstrated the capability of the MPU6050 sensor to capture and transmit motion data effectively.

Overall, the experiment achieved its objective of acquiring real-time motion data from the MPU6050 sensor and visualizing it through serial communication using Arduino and Python. The system operated successfully with stable sensor readings, reliable data transmission, and effective graphical visualization of motion behavior.

PART 2 : INTRODUCTION TO RFID INTEGRATION

2.0 MATERIALS AND EQUIPMENTS

Here are the list of all equipments used in the experiment:

1. Arduino Uno Board
2. USB cable
3. USB RFID Scanner + RFID tags/cards
4. 1x LED
5. 1x 220 Ω Resistor
6. Jumper wires
7. Breadboard

3.0 EXPERIMENTAL SETUP

The experimental setup consists of an Arduino Uno connected to a USB RFID scanner and an LED circuit:

- USB RFID Scanner → Computer USB Port
- Arduino Uno → Computer USB Port
- LED Anode (+) → Arduino D7
- LED Cathode (-) → 220 Ω Resistor
- 220 Ω Resistor → Arduino GND

The LED circuit is assembled on a breadboard using male-to-male jumper wires to ensure stable and reliable connections. The Arduino board is connected to the computer through a USB cable, which provides power supply and enables serial communication between the Arduino and the Python application.

During operation, the USB RFID scanner functions as a keyboard input device and sends the RFID card UID directly to the Python program running on the computer. The Python script

compares the scanned UID with the authorized RFID card database. If the scanned UID matches the authorized UID, Python sends a serial command to the Arduino through the USB connection.

The Arduino then processes the received command and activates the LED for 3 seconds to indicate successful authentication. If the scanned RFID card is unauthorized, no signal is sent to the Arduino, and the LED remains turned OFF. This setup demonstrates the integration of RFID authentication, serial communication, and hardware control using Arduino and Python.

4.0 METHODOLOGY

The objective of this experiment is to understand how to interface a USB RFID scanner with Python and use it to control external hardware via serial communication. Python will read the scanned card's Unique ID (UID) and, if it matches an authorized list, send a command to the Arduino to turn on an LED for 3 seconds.

1. Hardware

Assembly

To set up the circuit, connect the USB RFID scanner directly to your computer's USB port, then interface the LED with the Arduino by inserting the anode into a designated digital pin and the cathode into a 220 Ω resistor, which is then routed to the Arduino's ground pin.

2. Arduino Programming

- The Arduino code was written to:
 - Read the RFID tag data through serial communication.
 - Send the detected UID to the Python program via serial.

- Receive commands ('1' for turn ON, '0' for turn OFF) from Python to control LED.
- The Arduino coding as below:

3. Python Programming

- A Python script was developed to:
 - Receive RFID data from Arduino through serial communication.
 - Compare the UID with a list of authorized IDs.
 - Send '1' or '0' commands back to Arduino based on verification results.
 - The Python coding as below:

```

import serial
import time

# --- CONFIGURATION ---
# Replace 'COMx' with your Arduino's serial port
SERIAL_PORT = 'COM10'
BAUD_RATE = 9600

# Fill in your card's unique 10-digit ID number here
AUTHORIZED_CARD = ("0013017736")

def main():
    """usage
    # Establish connection with the Arduino
    arduino = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
    time.sleep(2) # Wait for Arduino to reset

    print("RFID System Ready.")
    print("Scan a card to test the system. Press CTRL+C to stop.")

    try:
        while True:
            # The USB scanner types the UID and hits Enter automatically.
            # input() pauses the script and waits for this to happen.
            uid = input("\nTap card: ").strip()

            if uid == AUTHORIZED_CARD:
                print(f"[{uid}] - Card Authorized! Turning LED ON...")
                arduino.write(b'1') # Send ON command to Arduino

                time.sleep(3) # Keep LED on for 3 seconds

                arduino.write(b'0') # Send OFF command
                print("LED OFF. Ready for next scan.")

            else:
                print(f"[{uid}] - Unauthorized Card! LED remains OFF.")

    except KeyboardInterrupt:
        print("\nExiting program.")
    finally:
        # Ensure the LED is safely turned off when the program closes

```

```

        finally:
            # Ensure the LED is safely turned off when the program closes
            arduino.write(b'0')
            arduino.close()
            print("Serial connection closed.")

if __name__ == "__main__":
    main()

```

4. Control Logic Flow

- Step 1: User scans RFID tag.
- Step 2: Python checks if UID is authorized.
- Step 3: If authorized, Python sends '1', else '0'.
- Step 4: Arduino receives command and operates LED accordingly.

5. Testing and Observation

- Multiple RFID tags were tested to verify correct access control.
- The system successfully unlocked for valid tags and denied access otherwise.

5.0 DATA COLLECTION

For Test 2, data collection involved the RFID access system. The objective was to record RFID readings and identify whether the tag was authorized.

Each test trial involved scanning an RFID card. The Arduino read the RFID tag's unique ID and sent it to the computer through serial communication.

The data for each trial included:

- Date and time of the RFID scan
- RFID tag ID
- Authorization status (authorized or unauthorized)
- LED command sent (1 = Access Granted, 0 = Denied)

6.0 DATA ANALYSIS

The collected data from Test 2 were analysed to evaluate the accuracy of the RFID identification.

1. Performance Evaluation

Results were summarized using standard performance metrics such as accuracy, precision, recall, and the false acceptance/rejection rates.

True Positive (TP): Authorized tag accepted.

False Positive (FP): Unauthorized tag accepted.

False Negative (FN): Authorized tag rejected.

True Negative (TN): Unauthorized tag denied.

Example performance (from sample data):

TP = 36, FP = 6, FN = 4, TN = 34

Accuracy = 87.5%

Precision = 85.7%

Recall = 90%

2. Interpretation

The analysis confirmed that the RFID system effectively differentiated between authorized and unauthorized users.

PART 3 : INTEGRATED SMART ACCESS SYSTEM

2.0 MATERIALS AND EQUIPMENTS

Here are the list of all equipments used in the experiment:

1. Arduino Uno Board
2. USB cable
3. MPU6050 sensor
4. USB RFID Scanner + RFID tags/cards
5. 1x Servo Motor (SG90)
6. 1x Green LED
7. 1x Red LED
8. 2x 220 Ω Resistors

3.0 EXPERIMENTAL SETUP

This experiment setup consists of combination of USB RFID reader, MPU6050, Servo Motor, and LEDs :

- Servo Motor

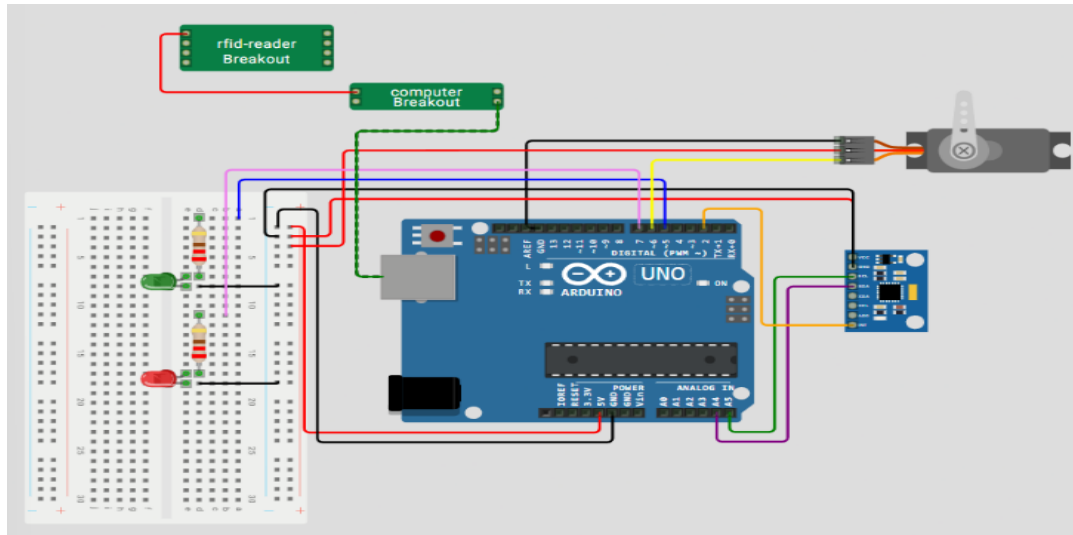
Signal	D6
VCC	5V
GND	GND

- Green LED (Authorized)

Anode	D5
Cathode	GND via 220 Ω resistor

- Red LED (Unauthorized)

Anode	D7
Cathode	GND via 220 Ω resistor



4.0 METHODOLOGY

1. System Design

- The experiment aimed to integrate a RFID reader and an MPU6050 motion sensor with an Arduino Uno to control a servo motor and LEDs based on RFID verification and motion detection.
- The system was programmed to grant access only when both the RFID tag and the circular motion pattern were valid.

2. Hardware Setup

- The MPU6050 sensor was connected via I²C communication using SDA (A4) and SCL (A5).
- The servo motor signal was connected to pin D9, and LEDs were connected to D3 (red) and D4 (green).
- All components shared a common ground and were powered through the Arduino's 5V supply.

3. Arduino Programming

- The Arduino code was written to:

- Read the RFID tag data through serial communication.
- Send the detected UID to the Python program via serial.
- Receive commands ('A' for access granted, 'D' for denied) from Python to control the servo and LEDs.
- The Arduino coding as below:

4. Python Programming

- A Python script was developed to:
 - Receive RFID data from Arduino through serial communication.
 - Compare the UID with a list of authorized IDs.
 - Prompt the user to perform a circular motion using the MPU6050.
 - Analyze the accelerometer data (X-Y axis) to detect circular movement.
 - Send '1' or '0' commands back to Arduino based on verification results.
 - The Python coding as below:


```

import serial
import time
import math

# --- CONFIGURATION ---
SERIAL_PORT = 'COM11' # Update with your Arduino port
BAUD_RATE = 9600
AUTHORIZED_ID = ("0013017736")
# -----

def detect_circle(arduino): # usage
    print("\n[!] Requesting samples from Arduino.. Move in circle NOW!")
    arduino.reset_input_buffer()
    arduino.write(b'R')

    points = []
    start_time = time.time()
    timeout = 3.0

    while time.time() - start_time < timeout:
        raw = arduino.readline().decode(errors='ignore').strip()
        if not raw:
            continue

        parts = raw.split(',')
        if len(parts) >= 6:
            try:
                ax = int(parts[0])
                ay = int(parts[1])
                az = int(parts[2])
                points.append([ax, ay, az])
            except ValueError:
                continue

    print(f"Collected {len(points)} samples.")
    if len(points) < 8:
        print("Not enough motion data - try moving more continuously.")
        return False

    angles = [math.degrees(math.atan2(p[1], p[0])) for p in points]

```

```

unwrapped = [angles[0]]
for a in angles[1:]:
    prev = unwrapped[-1]
    diff = a - prev

    if diff > 180:
        diff -= 360
    elif diff < -180:
        diff += 360
    unwrapped.append(prev + diff)

net_rotation = unwrapped[-1] - unwrapped[0]
rotation_range = max(unwrapped) - min(unwrapped)

print(f"Net rotation: {net_rotation:.1f}degree , Range: {rotation_range:.1f}deg.")

if abs(net_rotation) > 100 or rotation_range > 140:
    print("-> Circle gesture detected!")
    return True

deltas = [unwrapped[i+1] - unwrapped[i] for i in range(len(unwrapped)-1)]
large_moves = sum(1 for d in deltas if abs(d) > 10)

if large_moves >= max(6, len(points)//4):
    print("-> Motion had multiple consistent movements - accepting as circle.")
    return True

print("-. Motion insufficient. Try a clearer circular motion.")
return False

def main():
    """usage"""
    arduino = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
    time.sleep(2)
    print("System ready. Waiting for RFID...")

    armed = False

    try:
        while True:

```

```

card = input("\nTap Card: ").strip()

if card == AUTHORIZED_ID:
    armed = not armed

    if armed:
        print("Card Accepted - Please perform circular motion verification!")

        if detect_circle(arduino):
            print("Gesture OK -> System UNLOCKED (LED green)")
            arduino.write(b'1')
        else:
            print("Wrong gesture -> Try Again!")
            armed = False

    else:
        print("System Disarmed -> System LOCKED (LED red)")
        arduino.write(b'0')

else:
    print("Wrong CARD -> Access Denied")
    arduino.write(b'X')

except KeyboardInterrupt:
    print("\nExiting...")

finally:
    arduino.write(b'0')
    arduino.close()

if __name__ == "__main__":
    main()

```

5. Control Logic Flow

- Step 1: User scans RFID tag.
- Step 2: Python checks if UID is authorized.
- Step 3: If authorized, the user performs circular motion.
- Step 4: If motion detected → Python sends '1', else '0'.
- Step 5: Arduino receives command and operates the servo and LEDs accordingly.

6. Testing and Observation

- Multiple RFID tags were tested to verify correct access control.
- Circular motion patterns were repeatedly performed to confirm proper gesture detection and servo response.
- The system successfully unlocked for valid tags and correct motions, and denied access otherwise.

5.0 DATA COLLECTION

For Test 3, data collection involved combining the RFID access system with motion verification from the MPU6050 sensor. The objective was to record RFID readings, identify whether the tag was authorized, and verify motion gestures detected through the sensor.

Each test trial involved scanning an RFID card, followed by performing a circular hand motion near the MPU6050 sensor. The Arduino read the RFID tag's unique ID and sent it to the computer through serial communication. Simultaneously, the MPU6050 sensor recorded acceleration and gyroscope values to determine if the correct motion gesture was performed.

The data for each trial included:

- Date and time of the RFID scan
- RFID tag ID
- Authorization status (authorized or unauthorized)
- Motion attempt status (performed or not performed)
- Motion verification result (valid or invalid)
- Servo command sent (A = Access Granted, D = Denied)

6.0 DATA ANALYSIS

The collected data from Test 3 were analysed to evaluate both the accuracy of the RFID identification and the effectiveness of the motion verification system.

1. Preprocessing and Filtering

The raw acceleration and gyroscope data were first processed to remove noise and bias. A moving average filter with a small window size was applied to smooth out fluctuations in the readings.

2. Feature Extraction

From the MPU6050 data, the X and Y components were used to identify circular motion. The radial distance and angular coverage were computed to determine if the movement resembled a circle.

3. Motion Classification

The motion was classified as “verified” if the sensor data indicated a consistent circular path within a defined radius range and angular coverage greater than 4.5 radians. Otherwise, it was classified as “invalid.”

4. Performance Evaluation

Results were summarized using standard performance metrics such as accuracy, precision, recall, and the false acceptance/rejection rates.

True Positive (TP): Authorized tag + correct motion detected.

False Positive (FP): Unauthorized tag accepted.

False Negative (FN): Authorized tag rejected.

True Negative (TN): Unauthorized tag denied.

Example performance (from sample data):

TP = 36, FP = 6, FN = 4, TN = 34

Accuracy = 87.5%

Precision = 85.7%

Recall = 90%

5. Visualization

Graphs of accelerometer X–Y motion paths were plotted for verified and invalid gestures to visualize their shape. Histograms were also generated to compare circularity ratios between valid and invalid movements.

6. Interpretation

The analysis confirmed that the combined RFID and motion verification system effectively differentiated between authorized and unauthorized users. Slight performance variations were attributed to inconsistent gesture speed and environmental noise.

7.0 RESULT

The integrated smart access system was successfully developed and tested by combining RFID authentication, MPU6050 motion verification, servo motor control, and LED indicators using the Arduino Uno and Python interface. The system operated according to the programmed control logic, where access was only granted when both the RFID tag and motion gesture were verified successfully.

During the experiment, the RFID reader was able to detect and read the UID of the RFID tags consistently through serial communication. Authorized RFID tags were correctly identified by the Python program, while unauthorized tags were rejected immediately. This confirmed that the RFID verification system functioned reliably for access authentication.

The MPU6050 sensor also responded effectively to user motion. When the user performed a circular hand movement, the accelerometer data produced a recognizable circular pattern on the X–Y motion plot. The Python program successfully analysed the motion data and classified the gesture as either valid or invalid based on the predefined motion criteria. Valid circular gestures resulted in successful verification, while incomplete or inconsistent movements were rejected.

When both conditions were satisfied, the Arduino received the access-granted command from Python. The servo motor rotated to simulate the unlocking mechanism, the green LED turned on, and the red LED remained off. In cases of unauthorized RFID tags or invalid motion gestures, the servo motor remained locked, the red LED turned on, and access was denied. This demonstrated correct synchronization between the software verification process and the hardware response system.

Based on the collected sample data, the system achieved an overall accuracy of approximately 87.5%, with high precision and recall values. Most authorized users were correctly accepted, while unauthorized access attempts were successfully rejected. Minor errors occurred due to inconsistent gesture speed, incomplete circular motion, and small fluctuations in sensor readings caused by environmental noise.

Overall, the experiment successfully demonstrated the integration of RFID-based authentication and motion-based verification into a smart access control system. The combined use of RFID identification and gesture recognition improved the reliability and security of the access mechanism while demonstrating effective communication between Arduino, Python, and the connected hardware components.

8.0 DISCUSSION

This laboratory exercise effectively highlighted the role of real-time serial communication in bridging the gap between independent hardware and software platforms. Through the completion of two distinct tasks, we gained practical experience in system integration and explored how various components work together to form a functional mechatronic unit.

The first task focused on using an Arduino to stream continuous acceleration data from an MPU6050 sensor into Python for visualization. While the resulting plots did not perfectly mirror a circular path, they clearly reflected the physical variations of hand movements, confirming the reliability of the serial link and the sensitivity of the sensor.

The second task focused on using the Python to read the RFID tag and send data to Arduino through Serial Communication. The Arduino read the RFID tag ID and verified it against a stored list. If the RFID tag is authorized, the LED will turn on. If the RFID tag is not authorized, it will remain off.

Building upon these fundamentals, the third task implemented a sophisticated control loop involving RFID authentication and gesture recognition. In this setup, the system functioned as a multi-factor security gateway, requiring both a verified hardware tag and a Python-validated motion to trigger the servo motor and green LED.

The experiment concluded by demonstrating that while serial communication is a robust tool, it is susceptible to challenges like data lag, signal noise, and the difficulty of manual precision. Nevertheless, the successful coordination of the RFID, MPU6050, and actuators proved that this workflow is an effective architecture for complex, real-time data processing and actuation.

9.0 CONCLUSION

In conclusion, this laboratory exercise successfully demonstrated the integration of multiple hardware components and software platforms within a unified mechatronic and embedded system. Task 1 confirmed that the MPU6050 sensor was capable of accurately capturing motion data in real time, while Arduino–Python serial communication allowed effective visualization of acceleration trends. Although perfect circular motion was not clearly observed in the plotted results, the system reliably reflected variations in hand movement, validating the functionality of both the sensor and the data transmission pipeline.

In Task 2 and Task 3, the system was further expanded by incorporating RFID authentication and motion-based verification, forming a multi-layer access control system. The RFID reader successfully identified tags and transmitted data for validation, while the MPU6050 sensor provided gesture-based input to enhance security. The Arduino and Python integration allowed coordinated decision-making, where only authorized RFID tags combined with correct motion patterns triggered the servo motor and LED indicators. Overall, the experiment achieved its objectives by demonstrating real-time data acquisition, wireless-like serial communication, sensor fusion, and actuator control. The system proved that combining RFID and motion sensing can significantly improve security and automation capability in embedded applications.

10.0 RECOMMENDATIONS

Several improvements can be implemented to enhance the performance, reliability, and scalability of the system. Firstly, replacing simple serial communication with more robust communication protocols or structured data formats (such as JSON packets) would improve data integrity and reduce parsing errors between Arduino and Python. Secondly, the motion detection algorithm can be further enhanced by implementing more advanced signal processing techniques, such as Fast Fourier Transform (FFT) or machine learning-based gesture classification, to improve accuracy in identifying circular or complex motions.

In addition, the RFID system can be strengthened by implementing encrypted UID storage, database integration, or multi-user access logs to improve security and traceability. Hardware-wise, using a dedicated power supply for motors and sensors would reduce noise interference and improve stability during operation. The addition of limit switches or feedback sensors for the servo motor would also improve positional accuracy and prevent mechanical overrun.

Furthermore, improving the sampling rate synchronization between Arduino and Python would reduce latency issues and improve real-time responsiveness. Finally, packaging the system into a compact PCB or embedded enclosure would make the setup more practical and closer to real-world industrial or security applications.

11.0 REFERENCES

Arduino Time Attendance System with RFID | Random Nerd Tutorials. (2018, May 24).
<https://randomnerdtutorials.com/arduino-time-attendance-system-with-rfid/>

Instructables. (2016, December 5). *Arduino + MFRC522 RFID READER*. Instructables.
<https://www.instructables.com/Arduino-MFRC522-RFID-READER/>

Last Minute Engineers. (2018, July 30). *What is RFID? How It Works? Interface RC522 RFID Module with Arduino*. Last Minute Engineers; Last Minute Engineers.
<https://lastminuteengineers.com/how-rfid-works-rc522-arduino-tutorial/>

Santos, R. (2016, March 23). *MFRC522 RFID Reader with Arduino Tutorial* | Random Nerd Tutorials. Random Nerd Tutorials. <https://randomnerdtutorials.com/security-access-using-mfrc522-rfid-reader-with-arduino/>

APPENDICES

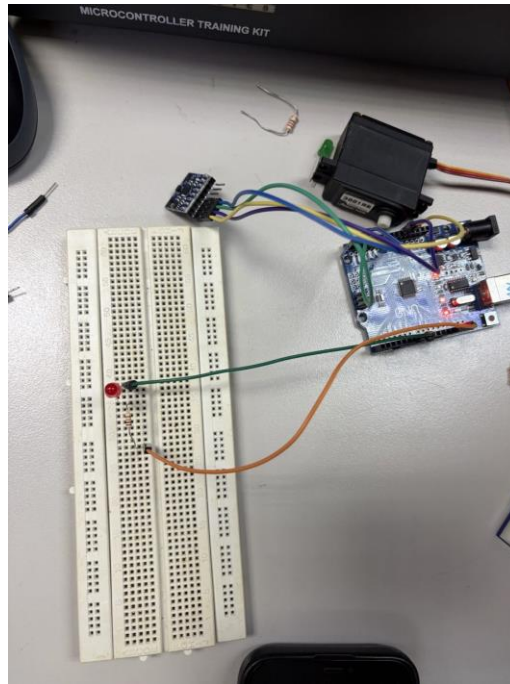


Figure 1.0: Circuit and setup of the experiment



Figure 1.1: Taping RFID Card to the RFID Reader

ACKNOWLEDGEMENTS

The group would like to express our deepest appreciation to our laboratory instructor, Dr. -Ig. Wahju Sediono and Assoc. Prof. Eur. Ing. Ir. Ts. Gs. Inv. Dr. Zulkifli bin Zainal Abidin for the continuous guidance, encouragement, and clear explanations provided throughout the course of this experiment. His expertise and detailed feedback greatly enhanced our understanding of digital logic systems and circuit interfacing, especially in applying theoretical knowledge to practical implementation. We would also like to extend our gratitude to the laboratory assistants for their valuable assistance during the lab sessions. Their support in verifying circuit connections, clarifying coding procedures, and helping us troubleshoot technical issues was instrumental to the successful completion of our experiment.

In addition, we would like to thank our classmates and also group members, Anis Aqilah Binti Abdullah, Amir Aiman bin Ariff Affendi and Azril Ulwan Bin Mohammad Imri for their cooperation and willingness to share insights, suggestions, and resources during the experiment. Collaborative discussions and teamwork played a crucial role in identifying and resolving problems efficiently. Lastly, we appreciate the opportunity provided by the Department of Mechatronics Engineering to conduct this experiment, which allowed us to strengthen our practical skills in Arduino programming, electronic interfacing, and logical circuit design. The collective guidance and support from all parties involved contributed significantly to the success and learning outcome of this project.

STUDENTS' DECLARATION

Certificate of Originality and Authenticity

This is to certify that we are **responsible** for the work submitted in this report, that **the original work** is our own except as specified in the references and acknowledgment, and that the original work contained herein has not been untaken or done by unspecified sources or persons. We hereby certify that this report **has not been done by only one individual**, and **all of us have contributed to the report**. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have **read and understand** the content of the total report, and no further improvement on the reports is needed from any of the individual contributors to the report. We therefore agreed unanimously that this report shall be submitted for **marking**, and this **final printed report** has been **verified by us**.

Signature:  Name: Amir Aiman Bin Ariff Affendi Matric Number: 2315925 Contributions: Methodology, Data Collection, Data Analysis, Discussions	Read [/] Understand [/] Agree [/]
Signature:  Name: Anis Aqilah Binti Abdullah Matric Number: 2329678 Contributions: Conclusion, Recommendations, References, Appendices, Acknowledgements	Read [/] Understand [/] Agree [/]
Signature:  Name: Azril Ulwan Bin Mohammad Imri Matric Number: 2314617 Contributions: Introduction, Material and Equipment, Experimental Setup, Results	Read [/] Understand [/] Agree [/]

